

A powerful index.

9

数组

理论课程



福州大学
FUZHOU UNIVERSITY

内容要点

- 数组
 - 一维数组的声明与使用
 - 数组下标越界
 - 多维数组的声明与使用
 - 字符数组

目录

	1	内存的组织
	2	数组
	3	字符数组
	4	
	5	

内存中的数据

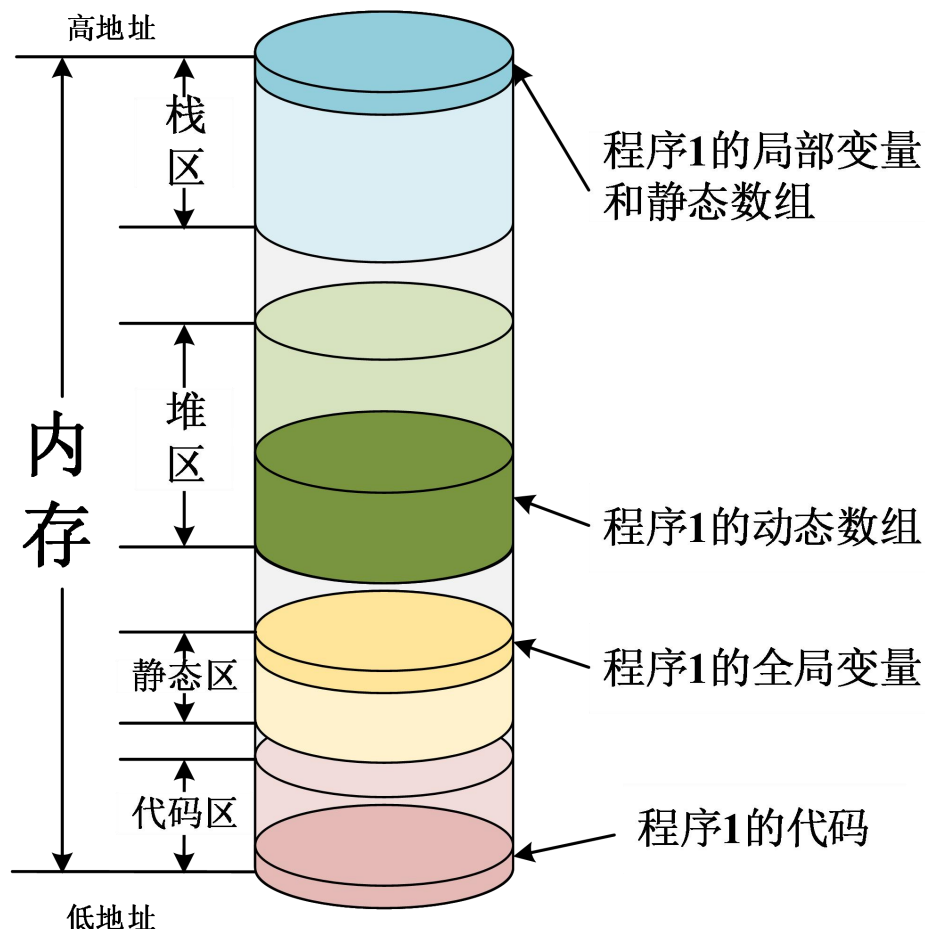
- 可执行程序指令和变量在运行时存储于内存中

- 内存四区

- 栈区 (Stack)
- 堆区 (Heap)
- 静态区 (Static)
- 代码区 (Code)

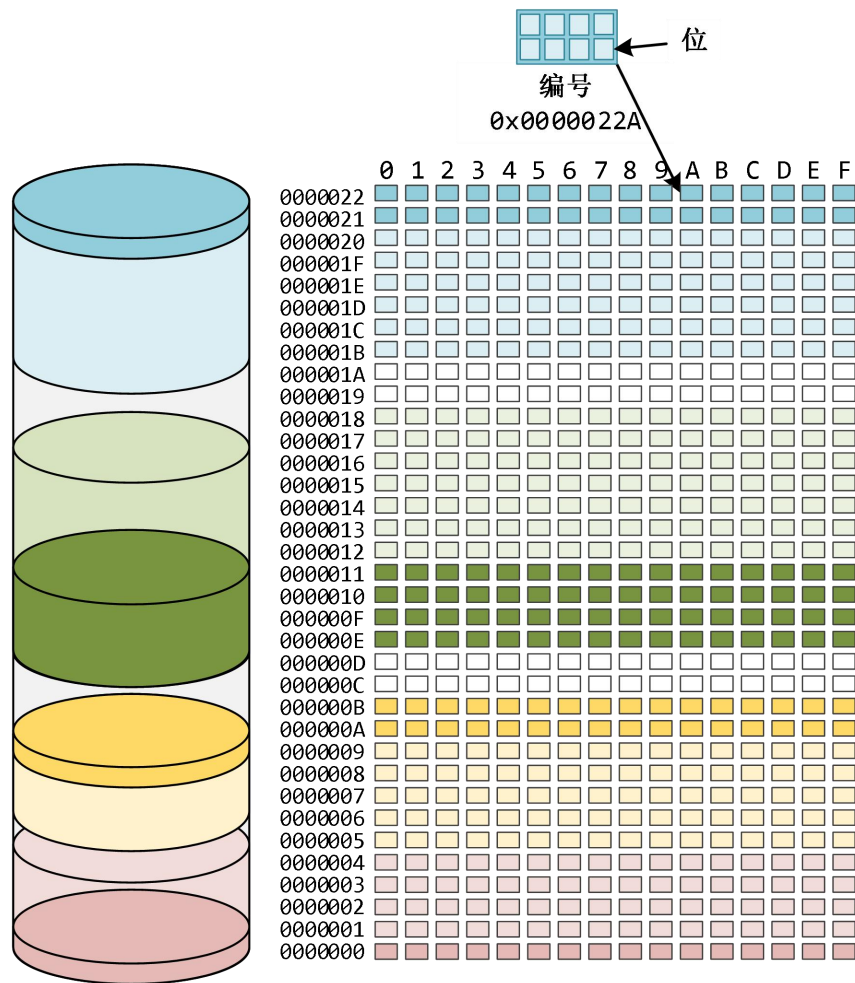
- 内存分配

- 内存的分配不是连续的



内存中的数据

- 位：处理数据的基本单位
 - 两种状态，值为 0 或 1
- 字节：存储的基本单位
 - 1字节折合8位，0—255的整数
 - 至少 $2^3=8$ 位足以表示英文字母
- 内存地址：对字节的编号
 - 最小值：0
 - 最大值：受各因素容量限制



内存地址

- 内存地址的最大值

指令字长是指机器指令中二进制代码的总位数。

- 由CPU、操作系统和应用程序的指令字长决定

- 64位应用程序无法运行于32位操作系统

- 64位操作系统无法运行于x86 CPU上

- 还受计算机实际内存大小的约束

CPU	操作系统	应用程序	内存范围	备注
32位	32位	32位	$0 - 2^{32} - 1$ B	64位系统无法运行于32位CPU
64位	32位	32位	$0 - 2^{32} - 1$ B	64位程序无法运行于32位系统
	64位	32位	$0 - 2^{32} - 1$ B	
		64位	$0 - 2^{64} - 1$ B	

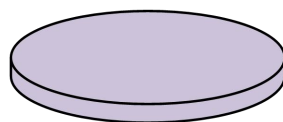
声明变量时的内存分配

- 执行声明语句时

```
int a;
```

- 内存栈区中开辟一个空间
- 从而变量具有

- 内存地址（偏移量）
- 占内存的长度
- 值（取决于长度和位的组织格式）



	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
000001E																
000001D																

000001EC							
000001ED							
000001EE							
000001EF							

- 栈区的空间用尽仍不够使用时，出现运行错误
- 堆区的空间用尽仍不够使用时，提示失败

目录

	2	数组
	2.1	数组声明和使用
	2.2	多维数组
	2.3	常见的编程错误
	3	字符数组

数组的声明

- 格式 `<类型> <数组名>[<数组长度>];`
 - 数组名的值 a 是声明时开辟的内存空间首地址，不可更改。
 - 声明时在类型前加注 `const`，数组元素不可更改。
 - 数组长度 N 应为无符号整型常数，可以为0。
 - 声明语句的方括号是修饰符；而表达式的方括号是操作符。
- 声明数组后开辟的空间：数组元素类型 $\times$ 数组长度
 - 设指定类型占有 t 字节，则开辟的空间为 tN 字节

声明语句	占用字节
<code>int arr[100];</code>	$4*100=400$
<code>double score[50], final_score[50];</code>	$8*50+8*50=800$
<code>const int base[3] = { 1, 2, 4 };</code>	$4*3=12$

数组的初始化

- 声明时，用复合文字初始化

```
int powers[8] = { 1, 2, 4, 6, 8, 16, 32, 64 };  
int powers[] = { 1, 2, 4, 6, 8, 16, 32, 64 };
```

- 数组长度以指定为准；不指定时，以复合文字长度为准

- 声明后，不可使用复合文字初始化

- 循环赋值（不一定要初始化为0）

```
for (i = 0; i < sizeof(arr) / sizeof(arr[0]); i++)  
    arr[i] = 0;
```

- 内存操作函数（注意内存格式）

```
memset(arr, 0, sizeof(arr));
```

- 数组应**先声明再使用**，数组元素应**先写后读**

复合文字（数组）

- 复合文字是按初始值设定项列表初始化的未命名对象。

- 类似于字符串、代码块，值为所占空间的首地址

```
char str[] = "Hello!";
```

72	101	108	108	111	33	0
----	-----	-----	-----	-----	----	---

```
int arr[] = { 1, 2, 4, 6, 8, 16, 32 };
```

1	2	4	6	8	16	32
---	---	---	---	---	----	----

- 格式：元素应为表达式，花括号为起止，逗号分隔

- 指定下标赋值 {1, 2, [4]=4, 6, [1]=16, 32}

1	16	32	0	4	6
---	----	----	---	---	---

- 显式指定下标的元素，以指定为准，后定义覆盖前定义
 - 未指定下标时，为左侧相邻元素下标加1（首元素下标为0）
 - 未显式指定值的元素默认为0
 - 数组长度为其元素中最大下标加1

```
// designate.c -- use designated initializers
```

```
#include <stdio.h>
```

```
#define MONTHS 12
```

```
int main(void)
```

```
{
```

```
int days[MONTHS] = { 31, 28, [4] = 31, 30, 31, [1] = 29 };  
int i;
```

```
for (i = 0; i < MONTHS; i++)  
    printf("%2d  %d\n", i + 1, days[i]);
```

```
return 0;
```

```
}
```

这是声明数组时使用复合文字进行初始化，用到了指定下标。

1	31
2	29
3	0
4	0
5	31
6	30
7	31
8	0
9	0
10	0
11	0
12	0

数组的元素访问

• 格式

运算符名称	格式	示例
下标运算符	<数组名>[<下标>]	arr[1] = 2;
间接寻址运算符	*(<数组名>+<下标>)	*(arr + 1) = 2;

– 数组下标 i 应为**整型表达式**，否则会有编译错误

- 可以为负数，甚至允许超过其边界（避免此做法）
- 数组越界将产生运行错误
- 下标 i 的下限是 0，上限是 $N-1$ ，超过即为越界

数组越界指的对数组元素的访问超过了边界。

• 下标和次序的关系

– 下标是偏移量，从0开始，第 $i+1$ 个元素对应下标为 i

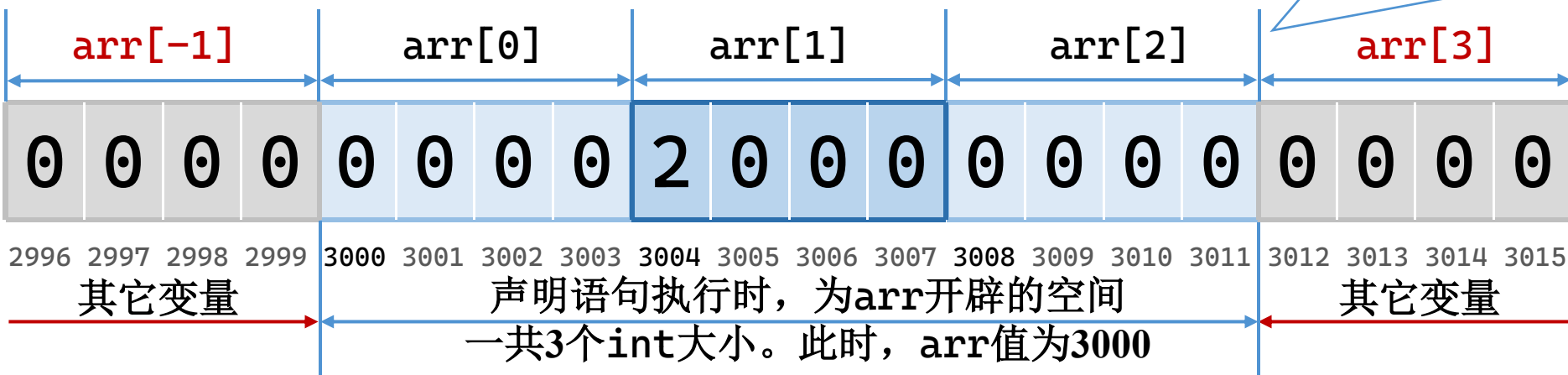
数组的元素

- 含义 `arr[index]`

- 元素的值是自 $a+it$ 至 $a+(i+1)t-1$ 字节所指内存区域的值
 - `arr` 的内存地址 a , `index` 为步数 i , 类型的长度 t (单位: 字节)
- 内存格式按元素类型解析

```
int arr[3]; arr[1]=2;
```

如果该区域是程序不可访问的, 将产生**运行错误**; 否则不会产生错误。



数组的遍历

- 通常使用for循环进行遍历
 - 从0到数组的元素个数

```
for (int i = 0; i < ARR_LENGTH; i++)
```

- 元素个数可以使用 `sizeof arr / sizeof arr[0]` 计算
 - 仅限于在数组名的作用域内


```
/* day_mon1.c -- prints the days for each month */
```

```
#include <stdio.h>
```

```
#define MONTHS 12
```

```
int main(void)
```

```
{
```

数组的声明

```
int days[MONTHS] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
```

```
int index;
```

for循环访问数组，一般以0开始，以“<数组长度”结束，下标增量

```
for (index = 0; index < MONTHS; index++)
```

```
    printf("Month %d has %2d days.\n", index + 1,  
          days[index]);
```

```
return 0;
```

数组的引用

```
}
```

Month 1 has 31 days.

Month 2 has 28 days.

Month 3 has 31 days.

(此处省略数行)

Month 12 has 31 days.

```
/* no_data.c -- uninitialized array */
```

```
#include <stdio.h>
```

```
#define SIZE 4
```

```
int main(void)
```

```
{
```

数组的声明不等于初始化

```
int no_data[SIZE]; /* uninitialized array */
```

```
int i;
```

```
printf("%2s%14s\n",  
       "i", "no_data[i]);
```

```
for (i = 0; i < SIZE; i++)
```

数组的元素不经初始化不可访问，否则结果不可靠

```
printf("%2d%14d\n", i, no_data[i]);
```

```
return 0;
```

```
}
```

i	no_data[i]
0	-858993460
1	-858993460
2	-858993460
3	-858993460

```
/* day_mon2.c -- letting the compiler count elements */
```

```
#include <stdio.h>
```

```
int main(void)  
{
```

有经验的程序员将不应修改元素的
数组标记为const来避免误改

```
    const int days[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31};  
    int index;
```

在数组声明范围内数组长度为
sizeof a / sizeof a[0]

```
    for (index = 0; index < sizeof days / sizeof days[0]; index++)  
        printf("Month %2d has %d days.\n", index + 1,  
               days[index]);
```

声明index，应按物理意义使用；不可写成：

```
    return 0;
```

for (index = 1; index <= LENGTH; index++)

```
}
```

Month 1 has 31 days.

Month 2 has 28 days.

(此处省略数行)

Month 10 has 31 days.

```
// bounds.c -- exceed the bounds of an array
```

```
#include <stdio.h>
```

```
#define SIZE 4
```

```
int main(void) {
```

```
    int value1 = 44;
```

```
    int arr[SIZE];
```

```
    int value2 = 88;
```

```
    int i;
```

```
    printf("value1 = %d, value2 = %d\n", value1, value2);
```

```
    for (i = -1; i <= SIZE; i++)
```

```
        arr[i] = 2 * i + 1;
```

```
    for (i = -1; i < 7; i++)
```

```
        printf("%2d %d\n", i, arr[i]);
```

```
    printf("value1 = %d, value2 = %d\n", value1, value2);
```

```
    printf("address of arr[-1]: %p\n", &arr[-1]);
```

```
    printf("address of arr[4]: %p\n", &arr[4]);
```

```
    printf("address of value1: %p\n", &value1);
```

```
    printf("address of value2: %p\n", &value2);
```

```
    return 0;
```

```
}
```

程序员不应越界访问数组，不应利用越界对数组上下的变量赋值。

value2		a[-1] a[0] a[1] a[2] a[3] a[4]										value1			
88	未知	未知	未知	未知	-1	1	3	5	7	9	未知	未知	未知	未知	44
B0	B4	B8	BC	C0	C4	C8	CC	D0	D4	D8	DC	E0	E4	E8	EC

002CFB+

Visual Studio C	Linux GCC
value1 = 44, value2 = 88	value1 = 44, value2 = 88
-1 -1	-1 -1
0 1	0 1
1 3	1 3
2 5	2 5
3 7	3 7
4 9	4 9
5 -858993460	5 5
6 44	6 0
value1 = 44, value2 = 88	value1 = 9, value2 = -1
address of arr[-1]: 002CFBC4	address of arr[-1]: 0028FED4
address of arr[4]: 002CFBD8	address of arr[4]: 0028FEE8
address of value1: 002CFBEC	address of value1: 0028FEE8
address of value2: 002CFBB0	address of value2: 0028FED4

value2		a[-1] a[0] a[1] a[2] a[3] a[4]										value1			
未知	未知	未知	未知	未知	-1	1	3	5	7	9	未知	未知	未知	未知	未知
C0	C4	C8	CC	D0	D4	D8	DC	E0	E4	E8	EC	F0	F4	F8	FC

0028FE+

断言

assert

功能	断言：在代码中捕捉程序员的假设或预期条件。	
格式	<code>void assert(int expression);</code>	
参数	<i>expression</i>	运行中预期为真的表达式。该式会被计算。
返回值	成功	无。
头文件	assert.h	
说明	1. 如果<i>expression</i>为真，或者当前非DEBUG模式，程序没有反应；否则，触发异常终止程序。 2. 如果<i>expression</i>有副作用，会生效。	

利用断言检查数组下标越界错误

• 原有程序

这里下标*i*受到for的限制，不可能越界，不需要判断

这里是*i*的表达式，未直接受到for的限制，可能越界，需要判断
数组下标越界往往是运行错误的原因。

```
#include <stdio.h>
#define ARR_LENGTH 10

int main() {
    int a[ARR_LENGTH];

    for (int i = 0; i < ARR_LENGTH; i++)
        a[i] = i * 2;
    for (int i = 0; i < ARR_LENGTH; i += 2)
        a[i] = a[i + 1] - a[i + 2];
    for (int i = 0; i < ARR_LENGTH; i++)
        printf("a[%d]=%d\n", i, a[i]);

    return 0;
}
```

利用断言检查数组下标越界错误

```
#include <stdio.h>
#include <assert.h>
#define ARR_LENGTH 10
int main()
{
    int a[ARR_LENGTH];
    for (int i = 0; i < ARR_LENGTH; i++)
        a[i] = i * 2;
    for (int i = 0; i < ARR_LENGTH; i += 2)
    {
        assert(i + 1 >= 0 && i + 1 < ARR_LENGTH);
        assert(i + 2 >= 0 && i + 2 < ARR_LENGTH);
        a[i] = a[i + 1] - a[i + 2];
    }
    for (int i = 0; i < ARR_LENGTH; i++)
        printf("a[%d]=%d\n", i, a[i]);
    return 0;
}
```

有经验的程序员制备大量测试数据，运行程序，如果在这里中止，说明该组数据下，数组下标越界。

数组的使用场景

- 数组的使用场景

- 数组是有序的元素序列。
- 物理意义相同，数据类型相同，仅有次序区别。

- 示例

- 计算一组学生的成绩
- 将一组数据以字节形式存储到文件
- 计算一幅图像像素的平均灰度值
- 向量、矩阵的计算

目录

	2	数组
	2.1	数组声明和使用
	2.2	多维数组
	2.3	常见的编程错误
	3	字符数组

多维数组的声明

- 格式 `<类型> <数组名>[<数组长度3>][<数组长度2>][<数组长度1>];`

- 以下以声明 `type m[Nd][Nd-1]...[N1]` 为例

- 数组长度 N_i 应为无符号**整型常数**，可以为0。

- 元素个数为各维度的乘积 $\prod_{i=1}^d N_i$

- 高维数组可以视为低维数组为元素的数组

- 较高的维度在前，较低的维度在后 `int matrix[3][5];`

这是3个int m[5]
堆起来的大数组

- 声明时用复合文字进行初始化

- 复合文字可以作为元素，组成大的复合文字。

```
int powers[2][8] = {{1,2,4,6,8,16,32,64},  
                    {1,2,4,6,8,16,32,64}};
```

多维数组的访问

- 格式 `<数组名>[<下标3>][<下标2>][<下标1>];`

- 以下以访问 $m[i_d][i_{d-1}]\dots[i_1]$ 为例

- 其中：数组下标应为**整型表达式**，否则会有编译错误

- 可以为负数，甚至超过其边界（程序员应避免此做法）

- 多维数组的应先计算偏移量再求值

$$\text{偏移量 } I = \sum_{k=2}^d \left(i_k \cdot \prod_{j=1}^{k-1} N_j \right) + i_1,$$

- d : 数组的维度数
- N_j : 第 j 维的长度（元素个数）
- i_k : 访问时第 k 维的下标值
- t : 每个元素的大小（字节数）
- 最后的 i_1 : 最低维的下标，直接加到总偏移量上

- 偏移量相同的，指向元素的值也相同

- 应该书写含义明确的下标形式

```
int m[3][5];
```

这是 $m[1][4]$ ，也是 $m[0][9]$ ，
 $m[2][-1]$, $m[-1][14]$, $m[3][-6]$

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----

```

/* rain.c  -- finds yearly totals, yearly average, and monthly
   average for several years of rainfall data */
#include <stdio.h>
#define MONTHS 12    // number of months in a year
#define YEARS 5      // number of years of data
int main(void) {
    // initializing rainfall data for 2010 - 2014
    const float rain[YEARS][MONTHS] = {
        {4.3,4.3,4.3,3.0,2.0,1.2,0.2,0.2,0.4,2.4,3.5,6.6},
        {8.5,8.2,1.2,1.6,2.4,0.0,5.2,0.9,0.3,0.9,1.4,7.3},
        {9.1,8.5,6.7,4.3,2.1,0.8,0.2,0.2,1.1,2.3,6.1,8.4},
        {7.2,9.9,8.4,3.3,1.2,0.8,0.4,0.0,0.6,1.7,4.3,6.2},
        {7.6,5.6,3.8,2.8,3.8,0.2,0.0,0.0,0.0,1.3,2.6,5.2}};
    int year, month;
    float subtot, total;
    printf(" YEAR      RAINFALL (inches)\n");
    for (year = 0, total = 0; year < YEARS; year++)
    {
        // for each year, sum rainfall for each month
        for (month = 0, subtot = 0; month < MONTHS; month++)
            subtot += rain[year][month];
        printf("%5d %15.1f\n", 2010 + year, subtot);
    }
}

```

```

        total += subtot; // total for all years
    }
    printf("\nThe yearly average is %.1f inches.\n\n", total / YEARS);
    printf("MONTHLY AVERAGES:\n\n");
    printf(" Jan  Feb  Mar  Apr  May  Jun  Jul  Aug  Sep  Oct ");
    printf(" Nov  Dec\n");
    for (month = 0; month < MONTHS; month++) {
        // for each month, sum rainfall over years
        for (year = 0, subtot = 0; year < YEARS; year++)
            subtot += rain[year][month];
        printf("%4.1f ", subtot / YEARS);
    }
    printf("\n");
    return 0;
}

```

YEAR	RAINFALL (inches)
2010	32.4
2011	37.9
2012	49.8
2013	44.0
2014	32.9

The yearly average is 39.4 inches.

MONTHLY AVERAGES:

Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
7.3	7.3	4.9	3.0	2.3	0.6	1.2	0.3	0.5	1.7	3.6	6.7

数组的局限性

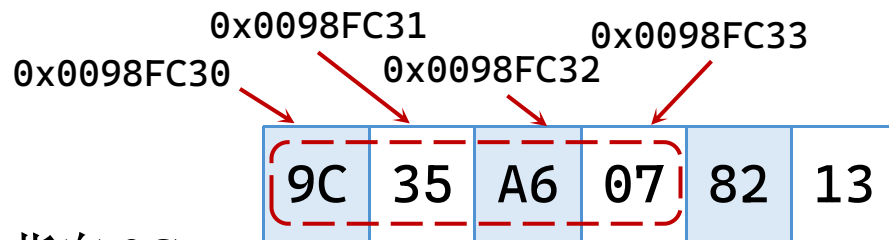
- 数组的局限性

- 数组作为局部变量时，所在栈区空间有限（默认1—8M）
- 数组作为全局变量时，破坏程序的逻辑结构
- 整理代码时，数组名是常量，无法被赋值

- 解决上述问题，需要新的数据类型：指针

- 指针为无符号整数，其物理意义为指向内存地址
- 指针的值为内存地址的偏移量，是内存一段区域的首地址
- 格式为“类型名”的类型、
长度为“类型名”的长度

- $p = 0x0098FC30$ 就意味着 p 指向 9C



数组的局限性

- 示例

- 比较2个字符串的首字母，将较大的字符串首置为大写

```
if (a[0] >= b[0])  
    a[0] = a[0] - 'a' + 'A';  
else  
    b[0] = b[0] - 'a' + 'A';
```

- 将字符串首置为大写字母，与字符串名无关

- 整理后的代码

```
if (a[0] >= b[0])  
    p = a;  
else  
    p = b;  
p[0] = p[0] - 'a' + 'A';
```

如果存在一种类型，作为能存储数组名的变量，那就更高内聚低耦合了。
这样的类型是指针。

目录

	2	数组
	2.1	数组声明和使用
	2.2	多维数组
	2.3	常见的编程错误
	3	字符数组

越界访问

- 越过数组边界，将产生运行错误

```
int table[4] = { 0 };  
int exists_in_table(int v) {  
    for (int i = 0; i <= 4; i++) {  
        if (table[i] == v) return 1;  
    }  
    return 0;  
}
```

当i循环至4时，数组越界。

越界访问

- 越过数组边界，将产生运行错误

```
int table[4][5] = { 0 };  
for (int i = 0; i < 4; i++) {  
    for (int j = 0; j < 5; j++) {  
        table[i][j]=i*j;  
    }  
}  
for (int i = 0; i < 4; i++) {  
    for (int j = 0; j < 5; j++) {  
        table[i][j]=table[i][j]+table[i][j+1];  
    }  
}
```

当j循环至5时，数组跳至下一行table[0][5]即table[1][0]，结果可能不及预期。
当i循环至4，j循环至5时，越界。

未初始化访问

- 访问未初始化的元素，结果不可控

```
int exists_in_table(int v) {  
    int table[4];  
    for (int i = 0; i < 4; i++) {  
        if (table[i] == v) return 1;  
    }  
    return 0;  
}
```

访问未初始化的元素，结果不可控。

目录

	1	内存的组织
	2	数组
	3	字符数组
	4	
	5	

字符和字符串

- 字符（char类型）

这里仅指ASCII码字符（char类型），现代程序包含汉字、日文、韩文等，则使用short类型，根据编码不同分为（Unicode、GB2312、Big5等码。）

- 可打印字符（0x20-0x7E）

- 字母、数字、运算符号、标点符号和其他符号

- 控制字符（0x00-0x1F，0x7F）

- 空格、回车、制表符、退格、警报等.....

- 字符串

- 一个有格式的字符数组，以数值为0的元素为结束。

- 物理意义：承载信息的文字

字符串的声明

• 字符串的声明

赋值方法	赋值语句	开辟空间	字符串常量开辟空间	复制数组
以数组形式声明	<code>char c[20];</code>	20B	0B	无初始化
在声明时赋值 (含长度)	<code>char c[20] = "Hello world!";</code>	20B	13B	自c[0]赋为'H'至c[11]赋为'!', c[12]赋为0, 其余赋为0。
在声明时赋值 (不含长度)	<code>char c[] = "Hello world!";</code>	13B	13B	
以指针变量形式赋值	<code>char *p = "Hello world!";</code>	4B/ 8B	13B	将字符串常量的内存首地址赋值给字符指针。

字符串的赋值

• 字符串的赋值

赋值方法	赋值语句
在声明时赋值	<code>char c[] = "Hello world!";</code>
通过字符串复制函数赋值	<code>strcpy(c, "Hello world!");</code>
通过字符串输入函数赋值	<code>scanf("%s" , c);</code>

• 注意事项

- 字符串数组名为常量，仅可在声明语句被字符串常量赋值
- 通过字符串函数赋值前应开辟足够的目标空间

字符串常量的其它形式

- 字符串常量（字符串文字）
 - 一对双引号中包含任何字符，特殊字符转义
 - 字符串允许中间断字（断行）
- 字符串存储时
 - 不包括双引号，包括双引号内的字符，添加结束标志（\0）

```
char greeting[50] = "Hello, and" " how are" " you"  
" today!";
```

```
char greeting[50] = "Hello, and how are you today!";
```

```

// talkback.c -- nosy, informative program
#include <stdio.h>
#include <string.h>
#define DENSITY 62.4
int main()
{
    float weight, volume;
    int size, letters;
    char name[40];
    printf("Hi! What's your first name?\n");
    scanf("%s", name);
    printf("%s, what's your weight in pounds?\n", name);
    scanf("%f", &weight);
    size = sizeof name;
    letters = strlen(name);
    volume = weight / DENSITY;
}

```

字符串函数的定义在这里

// for strlen() prototype

// human density in lbs per cu ft

密度 (DENSITY) 是62.4的物理意义, 有助于理清思路

字符串是字符数组

// name is an array of 40 chars

输入字符串的方法

求数组的空间

使用strlen()求字符串的长度

```
printf("Well, %s, your volume is %2.2f cubic feet.\n",  
name, volume);  
printf("Also, your first name has %d letters,\n",  
letters);  
printf("and we have %d bytes to store it.\n", size);  
  
return 0;  
}
```

通过%s打印字符串

Hi! What's your first name?

Sharla↵

Sharla, what's your weight in pounds?

139↵

Well, Sharla, your volume is 2.23 cubic feet.

Also, your first name has 6 letters,

and we have 40 bytes to store it.

字符串长度函数

strlen

功能	求字符串的长度，即首个空字符的下标。	
格式	<code>size_t strlen(const char* str);</code>	
参数	<code>str</code>	待检查的字符串指针
返回值	成功	字符串的长度，即空字符所在下标
头文件	<code>string.h</code>	
说明	1. 如果该字符串所在空间中不存在空字符，将越界继续寻找空字符。 2. 如果传入空指针，则返回0。	

字符串：长度和数组空间

- 字符串长度：`strlen`函数（应包含`<string.h>`）
 - 从数组起始位置开始数，到第一个数字0的偏移量。
- 字符串空间：`sizeof`操作符
 - 数组声明时，所分配的空间的字节数。

示例程序中为3
 - 字符串常量`sizeof(name)`比`strlen(name)`多1
 - 字符串变量`sizeof(name)`和`strlen(name)`无必然联系

字符形式	W	e	i	\0	*	*	*	越界
数值形式	87	101	105	0	*	*	*	越界
标记	起始			strlen				sizeof
索引	0	1	2	3	4	...	39	40

字符串函数

- 本节自习：注意输入、输出、作用
 - 长度: `strlen()`
 - 拼接: `strcat()`; `strncat()`
 - 比较: `strcmp()`; `strncmp()`
 - 复制: `strcpy()`; `strncpy()`
 - 打印: `sprintf()`
 - 其它: `strchr()`; `strpbrk()`; `strrchr()`; `strstr()`

谢谢观看

